Models of Phylogenetic Trees

Ferran Estrella, Gerard Carol, Arnau Ruiz, Bastiaan Hogervorst

December, 2025

Introduction

In this project, we study how evolution can be modelled using phylogenetic trees. We analyse the number of possible phylogenetic trees relating a given number of species, examine a tree reconstruction method, and develop techniques to compute the probability of observing a specific nucleotide distribution at the leaves of the tree, based on its topology and the transition matrices assigned to its edges.

Part 1: Tree topologies and Tree Reconstruction

1.1 Rooted Tree Topologies with Three Leaves

A *rooted phylogenetic tree* is a connected acyclic graph with a distinguished vertex called the *root*. The root represents the most recent common ancestor of all the leaves. In a rooted tree, every edge is oriented away from the root, determining an evolutionary direction from ancestors to descendants. A *leaf* is a vertex of degree 1 and usually represents an extant species (or taxon). A rooted tree is called *binary* if every internal node has exactly two children.

We are interested in counting how many rooted topologies are possible with three labelled leaves, which we denote by $\{A, B, C\}$. For three labelled leaves:

There are 4 rooted topologies in total, of which 3 are binary.

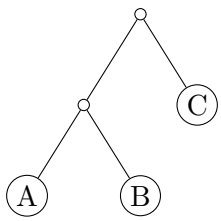
The reason is that a binary rooted tree on three leaves must first group two taxa together into a *cherry*, and there are exactly three possible pairs:

$$(A, B), \quad (A, C), \quad (B, C).$$

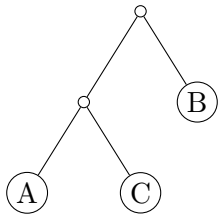
In addition, there is one non-binary (unresolved) topology, often called the *star tree*, in which all three leaves descend directly from the root.

Binary rooted trees

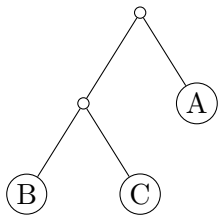
1. Topology $((A, B), C)$



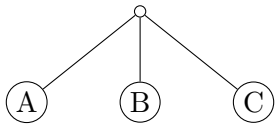
2. Topology $((A, C), B)$



3. Topology $((B, C), A)$



Non-binary rooted tree (star topology)



1.2 Interior Nodes and Edges in a Trivalent Unrooted Tree

A *trivalent* unrooted tree on n leaves is an unrooted tree in which every internal node has degree 3, and every leaf has degree 1.

Lemma 1. *Let n denote the number of leaves ($n \geq 3$). Given a trivalent unrooted tree of n leaves, the number I of internal nodes is equal to $n - 2$ and the number E of edges is equal to $2n - 3$.*

In order to prove it, we will use the well-know facts:

- A tree with V vertices has $E = V - 1$ edges.
- The sum of degrees of all vertices equals $2E$ (Handshake Lemma).

Proof. The total number of vertices is equal to the number of leaves plus the number of interior nodes, $V = n + I$, so the number of edges is

$$E = V - 1 = n + I - 1. \tag{1}$$

If we consider the sum of degrees counts, each leaf has degree 1 and each interior node has degree 3, thus

$$\text{sum of degrees} = 1 \cdot n + 3 \cdot I = 2E,$$

with the last equality being the Handshake Lemma. Hence, using :

$$2E = n + 3I = 2(n + I - 1),$$

we simplify:

$$n + 3I = 2n + 2I - 2 \quad \Longrightarrow \quad I = n - 2.$$

Thus the number of interior nodes is

$$I = n - 2.$$

To conclude, we compute the edges using $E = n + I - 1$:

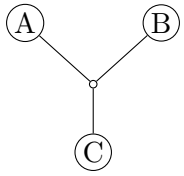
$$E = n + (n - 2) - 1 = 2n - 3,$$

so

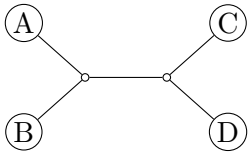
$$E = 2n - 3.$$

□

Case $n = 3$ (one interior node):



Case $n = 4$ (two interior nodes):



1.3 Unrooted Trivalent Tree Topologies

Let us present a result regarding the number of topologies of an unrooted trivalent tree.

Proposition 2. *Let $n \geq 3$ be the number of labelled leaves. The number of distinct labelled unrooted trivalent tree topologies with n leaves is*

$$T(n) \;=\; (2n - 5)!!, \tag{2}$$

where for an odd positive integer m the double factorial is $m!! = m \cdot (m - 2) \cdot (m - 4) \cdots 1$. Equivalently, putting $k = n - 2$,

$$T(n) = (2k - 1)!! = \frac{(2k)!}{2^k k!} = \frac{(2n - 4)!}{2^{n-2}(n - 2)!}.$$

Proof. To demonstrate the formula, we will use an induction argument. For $n = 3$ there is only one rooted binary tree with two labelled leaves, so $T_3 = 1$. Suppose, we have a tree with $n - 1$ leaves and we want to add a new leaf. From the last section, we know that given n leaves, we have $2n - 3$ edges. So if we had $n - 1$ leaves, we will have $2n - 5$ edges. Attaching the new leaf can be done as follows: First, subdivide the edge by inserting a new internal node in the middle of the edge. Then connect the new leaf to that node by a new edge, so maintaining the trivalent characteristic of the tree. By doing this, we added two edges more on our tree. At the end, one gets the recurrence:

$$T(n) = (2n - 5) \cdot T(n - 1), \quad n \geq 4$$

which gives us the double factorial formula

$$T(n) = \prod_{k=4}^{n} (2k - 5) = (2n - 5) \cdot (2n - 7) \cdot \cdots \cdot 3 \cdot 1 = (2n - 5)!!.$$

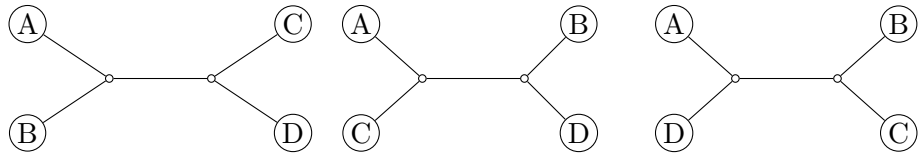
□

Small values and drawings. For reference:

$$T(3) = 1, \quad T(4) = 3, \quad T(5) = 15.$$

Below we draw all labelled unrooted trivalent topologies for $n = 4$ (there are three of them), and the two *unlabelled shapes* for $n = 5$ (there are many labelled versions — 15 total — obtained by permuting leaf labels).

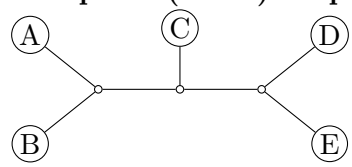
Case $n = 4$. There is a single unrooted trivalent *shape* (the quartet), but with labelled leaves there are $T(4) = 3$ distinct labelled trees. Here are three distinct labelings (A,B,C,D) of the quartet:



(Each of the three distinct pairings of the four leaf labels gives one labelled topology; there are 3 choices.)

Case $n = 5$. There are $T(5) = 15$ labelled unrooted trivalent trees. Up to isomorphism (ignoring labels) there is one non-isomorphic unrooted *shapes* on five leaves: the *caterpillar* (also called the "comb"). Note that relabelling the leaves in all possible ways, of course, produces the 15 labelled trees.

Caterpillar (comb) shape



General table for $T(n) = (2n-5)!!$ for $n = 3, \ldots, 20$. The following table lists the exact numbers (integer values) for labelled unrooted trivalent trees.

n	$T(n) = (2n-5)!!$
3	1
4	3
5	15
6	105
7	945
8	10 395
9	135 135
10	2 027 025
11	34 459 425
12	654 729 075
13	13 749 310 575
14	316 234 143 225
15	7 905 853 580 625
16	213 458 046 676 875
17	6 190 283 353 629 375
18	191 898 783 962 510 625
19	6 332 659 870 762 850 625
20	221 643 095 476 699 771 875

Asymptotic approximation (Stirling / double-factorial). We can approximate $T(n)$ using Stirling's approximation:

Put $k = n - 2$. Then

$$T(n) = (2k-1)!! = \frac{(2k)!}{2^k k!}.$$

Apply Stirling's approximation $m! \sim \sqrt{2\pi m}\,(m/e)^m$ to the numerator and denominator:

$$(2k)! \sim \sqrt{4\pi k}\,(2k/e)^{2k}, \qquad k! \sim \sqrt{2\pi k}\,(k/e)^k.$$

Hence

$$T(n) \sim \frac{\sqrt{4\pi k}\,(2k/e)^{2k}}{2^k\sqrt{2\pi k}\,(k/e)^k} = \sqrt{2}\left(\frac{2k}{e}\right)^k.$$

Re-substituting $k = n - 2$,

$$T(n) \;\sim\; \sqrt{2}\left(\frac{2(n-2)}{e}\right)^{n-2} \;\approx\; C\left(\frac{2n}{e}\right)^n$$

for some slowly varying constant C.

Interpretation of growth. The number of labelled unrooted trivalent trees grows *super-exponentially* in n; roughly it behaves like

$$\exp\big((n-2)\log(2(n-2)/e) + O(\log n)\big),$$

so—informally—about $\left(\frac{2n}{e}\right)^n$ up to subexponential factors. In particular, it is far larger than any exponential a^n with fixed a; it grows roughly like n^n (times the exponential factor $(2/e)^n$), i.e. super-exponential / "factorial-like" growth.

Remark on Bauer (2007). Bauer discusses accurate approximations for double factorials using refinements of Stirling's formula; applying such refinements yields the same leading behaviour as shown above and can give more accurate multiplicative constants if desired.

For instance, for the double factorial he uses the improved Stirling formula,

$$(2k-1)!! \sim \left(\frac{2n}{e}\right)^2 \sqrt{\frac{(12n+1)}{(6n+1)}}. \tag{3}$$

There is also a variant free of e which comes from:

$$\binom{2n}{n} \sim \frac{4^n}{\sqrt{\pi n}}, \qquad \text{or equivalently} \qquad \frac{(2n)!}{(n!)^2} \sim \frac{4^n}{\sqrt{\pi n}},$$

$$(2n-1)!! \;\sim\; \frac{2^n\, n!}{\sqrt{\pi n}}. \tag{3a}$$

Working a little one obtains

$$((2n-1)!!)^2 \;\sim\; \frac{(2n)!}{\pi n},$$

i.e., the slightly better variant:

$$(2n-1)!! \;\sim\; \frac{\sqrt{(2n)!}}{\sqrt{\pi n}}, \tag{3b}$$

where the denominator has to be positive. Bauer proves that the expression (3b) is better than the (3a) showing that the relative error is asymptotically $\frac{1}{8n}$ for the first expression while it is $\frac{1}{16n}$ for the second one. In fact he discusses that getting a more suitable δ_n there is a considerable improvement on the formulas (3a) and (3b) with now an asymptotically relative errors of $\frac{1}{(8n)^2}$ and $\frac{1}{128n^2}$, respectively.

1.4 Feasibility of Reconstruction Methods for African Origin of Humankind

For a phylogenetic tree with n labelled leaves, even very restrictive assumptions already lead to an enormous number of possible topologies. This shows immediately that any method attempting to examine all possible trees is computationally infeasible for $n = 147$.

Rooted Binary Trees

If we restrict attention to *rooted binary* trees, every internal node (including the root) has exactly two children. Let T_n denote the number of such trees with n labelled leaves. We can derive the classical result proved in the last section:

$$T_n = (2n-3)!! = 1 \cdot 3 \cdot 5 \cdots (2n-3).$$

For $n = 147$ this gives

$$T_{147} = (2 \cdot 147 - 3)!! = 291!!.$$

Approximation via Stirling's formula

To understand how enormous T_{147} is, we use Stirling's formula for factorials. Recall that for large m,

$$m! \sim \sqrt{2\pi m} \left(\frac{m}{e}\right)^m.$$

Now $(2n-3)!! = \frac{(2n-2)!}{2^{n-1}(n-1)!}$. For $n = 147$ one approximates both numerator and denominator via Stirling-type expressions, obtaining roughly

$$T_{147} \approx \frac{\sqrt{2\pi\,(2n-2)}\,\left(\frac{2n-2}{e}\right)^{2n-2}}{2^{\,n-1}\,\sqrt{2\pi\,(n-1)}\,\left(\frac{n-1}{e}\right)^{n-1}}.$$

Plugging $n = 147$, one finds that T_{147} is on the order of

$$\approx 4.9 \times 10^{296}.$$

Thus even under the very restrictive assumption of binary branching, the search space already contains on the order of 10^{297} trees.

Rooted vs. unrooted trees

Another way to see the size of the search space is to compare rooted and unrooted binary trees. The number of unrooted binary trees with n labelled leaves is $(2n-5)!!$. Each unrooted binary tree has exactly $2n-3$ edges, and placing a root on any edge produces a rooted binary tree. Hence,

$$\#\{\text{rooted binary trees}\} = (2n-3)\,(2n-5)!! = (2n-3)!!.$$

If we drop the binary restriction and allow multifurcations (internal degree ≥ 3), then *every* binary tree remains allowed, plus many additional topologies. Therefore the total number of rooted trees with 147 labelled leaves satisfies

$$\#\{\text{all rooted topologies with 147 leaves}\} \gg (2 \cdot 147 - 3)!! \approx 4.9 \times 10^{296}.$$

Computational feasibility

Suppose, unrealistically optimistically, that an algorithm could evaluate

$$10^9 \text{ tree topologies per second.}$$

Even checking only the binary rooted trees would require at least

$$\frac{10^{297}}{10^9} = 10^{288} \text{ seconds.}$$

Since one year is about 3×10^7 seconds, this corresponds to roughly

$$10^{281} \text{ years.}$$

The age of the universe is about 10^{10} years, so this is approximately

$$10^{271}$$

times the age of the universe. This is an immense *underestimate*, as we only counted binary trees and assumed a near-instantaneous evaluation per topology.

Conclusion

Even under the strongest restrictions (binary, rooted) the number of possible tree topologies for 147 labelled leaves is on the order of 10^{297}. Allowing more general rooted trees only increases this number. Therefore, any algorithm attempting to enumerate or evaluate all possible tree topologies is completely infeasible, and practical phylogenetic inference must instead rely on heuristic and statistical methods rather than exhaustive search.

1.5 Reconstruction of a Tree from a Compatible Set of Splits

In this section, we give a formal description of how a tree can be reconstructed uniquely from a complete and compatible set of splits. This includes formal definitions, a reconstruction algorithm and pseudocode

Preliminary Definitions

Definition 3 (Split)**.** Let X be a finite set of leaves. A *split* of X is a bipartition $U \mid V$ of X into two nonempty disjoint subsets with $U \cup V = X$ and $U \cap V = \emptyset$. A split is *trivial* if $\min\{|U|, |V|\} = 1$.

Definition 4 (Split Induced by an Edge)**.** Let T be an unrooted tree whose leaves are exactly the set X. Every edge e of T induces a split $U \mid V$ defined by the bipartition of X obtained by removing e from T. The set of all such splits is denoted $\mathcal{S}(T)$.

Definition 5 (Compatibility)**.** Two splits $U_1 \mid V_1$ and $U_2 \mid V_2$ on X are *compatible* if at least one of the intersections

$$U_1 \cap U_2, \quad U_1 \cap V_2, \quad V_1 \cap U_2, \quad V_1 \cap V_2$$

is empty. A set $\mathcal{S}$ of splits is *pairwise compatible* if every pair of splits in $\mathcal{S}$ is compatible.

Remark 6. Throughout this section we assume that the split set $\mathcal{S}$ is pairwise compatible. Compatibility is essential: if $\mathcal{S}$ contained two incompatible splits, then no tree T could satisfy $\mathcal{S}(T) = \mathcal{S}$. Thus compatibility is the necessary condition ensuring that the clusters form a hierarchical structure, which is precisely the structure encoded by an unrooted tree.

Reconstruction Algorithm

Let X be a finite set of leaves and let $\mathcal{S}$ be a pairwise compatible set of splits on X that contains all edge-induced splits of some unrooted tree T with leaf set X. To reconstruct the tree from $\mathcal{S}$ we start from a single split and then repeatedly "refine" an existing cluster whenever we encounter a split whose one side is a proper subset of that cluster. We will define $L(\cdot)$ as the function which maps a vertex to its associated leaves.

1. Choose an arbitrary split $U \mid V \in \mathcal{S}$. Create two vertices v_U and v_V joined by an edge. Associate the cluster U with v_U and the cluster V with v_V. So we have that $L(v_U) = U$ and $L(v_V) = V$

2. For the next remaining split $A \mid B \in \mathcal{S}$ (in an arbitrary order), select one of its sides, say $C \in \{A, B\}$, such that C is a subset of the leaves associated with one of the already existing vertices. So we have for one of the created vertices, say v that $C \subsetneq L(v)$.

3. If $C \subset L(v)$, introduce a new vertex u and add an edge between u and v. Set $L(u) := C$ and update the cluster of v to
$$L(v) := L(v) \setminus C.$$
Intuitively, this "peels off" the leaves in C from v and attaches them to a new vertex u.

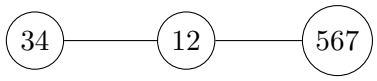
4. After all splits have been processed, each vertex v carries a cluster $L(v)$. Each leaf $x \in X$ ultimately appears exactly once in some $L(v)$; these vertices are then identified as the leaf vertices of the reconstructed tree.

Worked Example

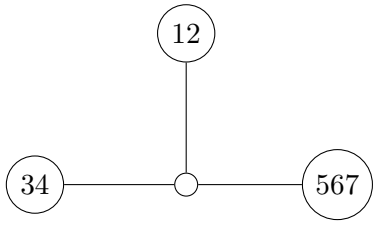
Now, let us consider an example to see how the algorithm works. Let
$$X = \{1, 2, 3, 4, 5, 6, 7\},$$
and consider the following collection of four (pairwise compatible) non-trivial splits:
$$\mathcal{S} = \Big\{\ 1234 \mid 567,\ 34 \mid 12567, 12 \mid 34567, 56 \mid 12347\Big\}$$
We will process these splits in the order they are given right now, but as stated previously, the order of processing does not affect the final reconstructed tree (up to isomorphism); it only changes the intermediate refinement sequence.

1. Initialize from an arbitrary split Start with the split $1234 \mid 567 \in \mathcal{S}$. Create two vertices joined by an edge and set
$$L(v_{1234}) = 1234, \qquad L(v_{567}) = 567.$$

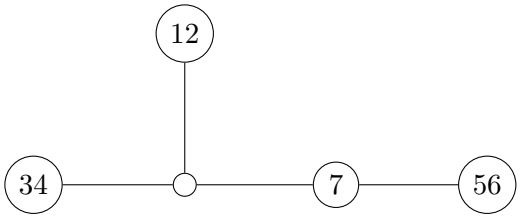
2. Process the split $34 \mid 12567$
Choose the side $C = 34$, which satisfies $34 \subset L(v_{1234}) = 1234$. Refine $L(v_{1234})$ into C and $L(v_{1234}) \setminus C = \{1, 2\}$: introduce a new vertex u with $L(u) = 34$, connect it to v_{1234}, and update $L(v_{1234}) \leftarrow 12$.



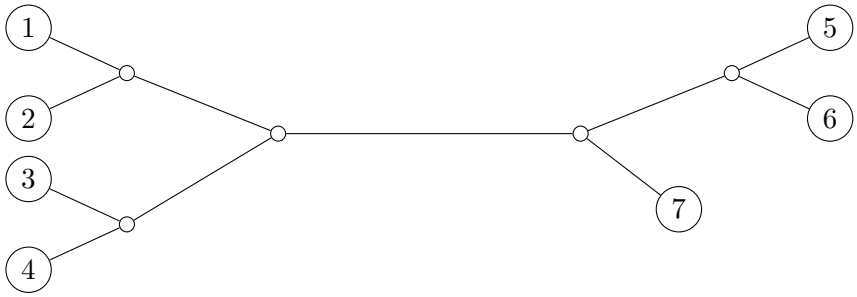
3. Process the split $12 \mid 34567$
   Select the side $C = 12$. Here we have $C = L(v)$ for the existing vertex labeled 12, so we refine by creating a new internal node (representing no leafs), as we have that $L(v_{12}) \setminus C = \emptyset$



4. Process the split $56 \mid 12347$
   Choose the side $C = 56$, which satisfies $56 \subset L(v_{567}) = 567$. Refine $L(v_{567})$ into 56 and $567 \setminus 56 = \{7\}$: introduce a new vertex t with $L(t) = 56$, connect it to v_{567}, and update $L(v_{567}) \leftarrow 7$.



5. Attach individual leaves and obtain the final tree
   At this point, singleton cluster 7 correspond directly to a leaf vertex. For non-singleton clusters 12, 34, and 56, we replace each such cluster by an internal vertex adjacent to each leaf in that cluster. This yields the final reconstructed unrooted tree on X:



   One can check that the four internal edges of the final tree induce exactly the splits in $\mathcal{S}$: $12 \mid 34567$, $34 \mid 12567$, $56 \mid 12347$, and $1234 \mid 567$.

Pseudocode for the Reconstruction Algorithm

We now give a pseudocode formulation of the reconstruction procedure described above.

Algorithm 1 Tree reconstruction from a compatible set of splits

Require: Pairwise compatible split set $\mathcal{S}$ on leaf set X.
Ensure: Tree T whose edge-induced splits are exactly $\mathcal{S}$.

```
Choose an arbitrary split U | V ∈ S.
Create two vertices v_U and v_V.
Create edge {v_U, v_V}.
Set L(v_U) ← U and L(v_V) ← V.
for each remaining split A | B ∈ S (in arbitrary order) do
    Find a vertex v and a side C ∈ {A, B} such that C ⊂ L(v).
    Create a new vertex u.
    Create edge {u, v}.
    Set L(u) ← C.
    Update L(v) ← L(v) \ C.
end for
For each leaf x ∈ X, attach a degree-1 leaf labelled x to the unique vertex v with x ∈ L(v).
return T =0
```

Part 2: Evolutionary Models on a Tree

In this part of the project, we introduce the mathematical structure that defines a rooted phylogenetic tree and construct evolutionary models on it. We use this formalism to find an expression for the probability of observing a certain nucleotide distribution at the leaves of a phylogenetic tree, and we apply it to various types of rooted trees, including rooted binary and trivalent phylogenetic trees. Moreover, we analyse the computational efficiency of the resulting expression and introduce Felsenstein pruning algorithm as a more efficient alternative. In the final section, we introduce the concept of branch length for an edge of a phylogenetic tree and relate it to both the log-det distance and the mutation rate.

Let us start with some basic definitions.

Definition 7 (Rooted tree)**.** A *rooted tree* is a directed acyclic graph, characterized by directed edges

$$E = \{\, e = (p \to c) \mid p \text{ is the (unique) parent of a child } c \,\}$$

and nodes of three types:

$$\begin{bmatrix} I \equiv \text{Interior node} \\ R \equiv \text{Root (interior node with no parent)} \\ L \equiv \text{Leaf } (\deg(L) = 1) \end{bmatrix}$$

The direction of the edges in the graph (i.e., the orientation of the graph) must be consistent with the root; that is, edges are directed from the root to the leaves.

Definition 8 (Rooted phylogenetic tree)**.** A *rooted phylogenetic tree* is a rooted tree endowed with the following structure:

- For each node: There exists a *discrete random variable* taking values on the set of nucleotides $\Sigma = \{A, C, G, T\}$:

  $$X : \Omega \to \Sigma.$$

  A realization of X is a *state* of the node, $x \in \Sigma$.

- For each edge: There exists a *transition matrix*:

  $$\mathbf{M}_e = \begin{pmatrix} P^e_{A\to A} & P^e_{A\to C} & P^e_{A\to G} & P^e_{A\to T} \\ P^e_{C\to A} & P^e_{C\to C} & P^e_{C\to G} & P^e_{C\to T} \\ P^e_{G\to A} & P^e_{G\to C} & P^e_{G\to G} & P^e_{G\to T} \\ P^e_{T\to A} & P^e_{T\to C} & P^e_{T\to G} & P^e_{T\to T} \end{pmatrix}$$

  which is a Markov matrix that, in general, depends on the edge and the nucleotide substitution model.

We can construct an **evolutionary model on a phylogenetic tree** as a *hidden Markov process*:

- *Hidden Process*: The particular states are *unknown* at the interior nodes and *known* at the leaves.

- *Markov Process*: The substitution process on an edge $e = (p \to c)$ only depends on the transition matrix $\mathbf{M}_e$ and the parent state:

  If $x_p \in \Sigma$ is a parent state and $x_c \in \Sigma$ is a child state, then

$$\Pr(X_c = x_c \mid X_p = x_p) = P^e_{x_p \to x_c} \equiv M_e(x_p, x_c).$$

  Hence, using the law of total probability, we have the expression:

$$\Pr(X_c = x_c) = \sum_{x_p \in \Sigma} \Pr(X_p = x_p)\, M_e(x_p, x_c)$$

Remark 9. In general, a rooted phylogenetic tree can have interior nodes with different numbers of descendants. However, one can assume, without loss of generality, that each interior node has exactly $d = 2$ descendants; that is, that phylogenetic trees are binary.

This is because, given any rooted phylogenetic tree, there always exists an **equivalent rooted binary phylogenetic tree**. Indeed, to find this equivalent binary tree, one simply considers each interior node I with d descendants and proceeds recursively as follows:

1. Consider a pair of edges $e_1 : I \to v_1$ and $e_2 : I \to v_2$ among the descendants of I.
2. Add a new edge $e : I \to I'$ such that I' is a new interior node with descendants v_1 and v_2 via the edges e_1 and e_2, and assign to e the identity transition matrix, $M_e = Id$.
3. Repeat this process for each remaining pair of edges among the descendants of I, until I has exactly two children.

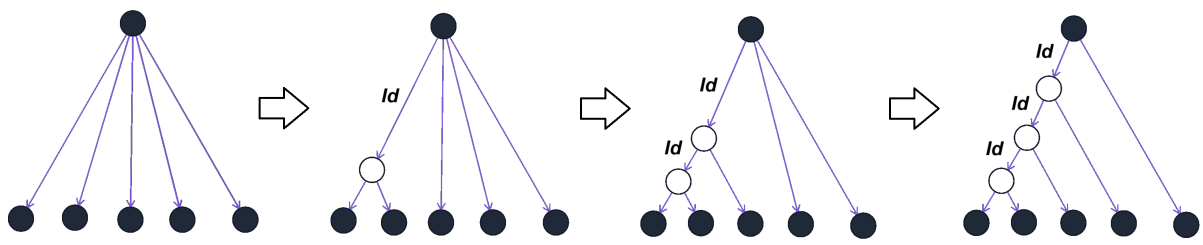


Figure 1: Schematic illustration of the recursion described above for an interior node with $d = 5$ descendants. Note that the nodes with empty interiors are the interior nodes added during the recursion, and that each edge whose child is one of these nodes has the identity matrix as its associated transition matrix.

2.1 Nucleotide Distribution Probabilities at the Leaves of a Phylogenetic Tree

In this theoretical framework, in which evolution is modelled through a rooted phylogenetic tree, we try to address the following problem:

> What is the joint probability of observing a given nucleotide distribution at the leaves of a phylogenetic tree?

That is, if $X_1, \ldots, X_n$ denote the random variables at the leaves of the tree, and we consider states $x_1, \ldots, x_n \in \Sigma$, we want to find:

$$\Pr(X_1 = x_1, \ldots, X_n = x_n) = \Pr(x_1, \ldots, x_n) \tag{4}$$

First, let us consider some particular cases in which we compute this joint probability on specific phylogenetic trees.

Rooted binary tree with three leaves: Let us consider a rooted binary tree with three leaves, a simple example we introduced before (Figure 2).

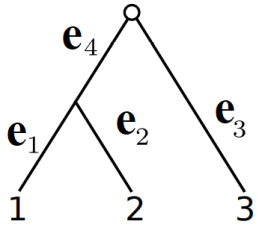


Figure 2: Rooted binary tree with three leaves, denoted by 1,2 and 3. The directed edges are labelled as $\mathbf{e}_i$ with $i = 1, 2, 3, 4$.

Let us assume that we have a particular nucleotide distribution at the leaves of the tree, for instance, C at the first leaf, another C at the second leaf, and an A at the third leaf, which has the root as its parent node. Then, we want to find:

$$\Pr(C, C, A) = \Pr(X_1 = C, X_2 = C, X_3 = A). \tag{5}$$

To compute the joint probability, we need to consider all possible nucleotide distributions on the whole tree that yield the considered nucleotide distribution CCA at the leaves. Since this tree has $I = 2$ internal nodes (including the root), the number of possible nucleotide distributions on the tree is $4^I = 16$, as shown in Figure 3.

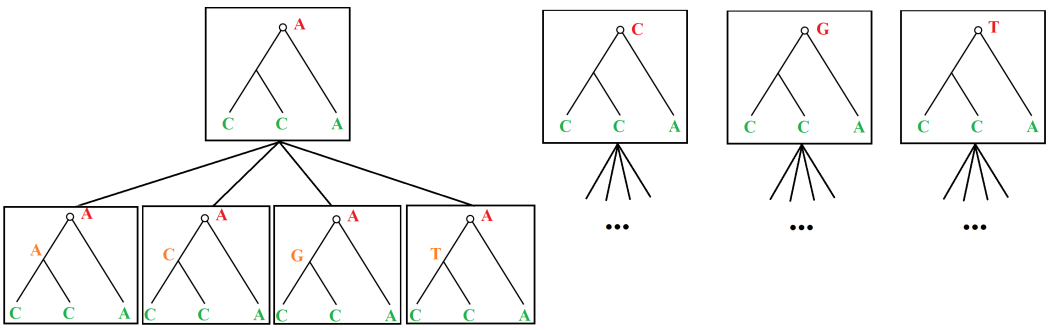


Figure 3: Schematic representation of the possible nucleotide distributions on the binary tree with three leaves that yield the configuration CCA at the leaves.

Therefore, the total probability (5) we want to compute is the sum of the probabilities of the 16 nucleotide distributions on the tree. Explicitly, it can be expressed as

$$\Pr(C, C, A) = \sum_{x_R \in \Sigma} \sum_{x_I \in \Sigma} P_R(x_R)\, M_{e_4}(x_R, x_I)\, M_{e_1}(x_I, C)\, M_{e_2}(x_I, C)\, M_{e_3}(x_R, A) \tag{6}$$

We note that we sum over all possible hidden states at the internal nodes of the tree, and for each possible nucleotide distribution on these internal nodes, we compute the probability by weighting with the root distribution, $P_R(X_R)$, and multiplying by the conditional probabilities of transitioning from each parent state to each child state, $M_e(x_p, x_c)$; for this tree, from $R \to I$, $I \to L_1, L_2$, and $R \to L_3$.

In general, the same expression can be applied to compute the joint probability of a nucleotide distribution XYZ at the leaves of the tree, simply by substituting the specific example CCA:

$$\Pr(X,Y,Z) = \sum_{x_R \in \Sigma} \sum_{x_I \in \Sigma} P_R(x_R)\, M_{e_4}(x_R, x_I)\, M_{e_1}(x_I, X)\, M_{e_2}(x_I, Y)\, M_{e_3}(x_R, Z) \tag{7}$$

Rooted binary tree with four leaves, balanced configuration: A similar reasoning can be applied to compute the joint probability of a nucleotide distribution $XYZT$ on a rooted binary tree with four leaves in a balanced configuration (Figure 4):

$$\begin{aligned}\Pr(X,Y,Z,T) = \sum_{x_R, x_{I_1}, x_{I_2} \in \Sigma} & P_R(x_R) M_{e_{I_1}}(x_R, x_{I_1}) M_{e_{I_2}}(x_R, x_{I_2}) \cdot \\ & \cdot M_{e_1}(x_{I_1}, X) M_{e_2}(x_{I_1}, Y) M_{e_3}(x_{I_2}, Z) M_{e_4}(x_{I_2}, T)\end{aligned} \tag{8}$$

Here, we note that we are summing over 4^3 possible nucleotide distributions at the internal nodes of the tree, since we now have a root and two additional internal nodes.

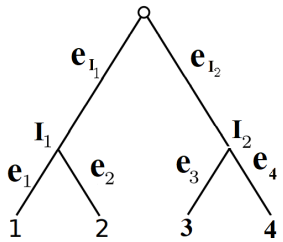


Figure 4: Rooted binary tree (in a balanced configuration) with four leaves, denoted by 1, 2, 3, and 4. The directed edges are labelled as $\mathbf{e}_i$, with $i = 1, 2, 3, 4$ for those whose children are the leaves, and $i = I_1, I_2$ for those whose children are the internal nodes.

General case: Finally, let us generalize the previous procedure to any phylogenetic tree.

Lemma 10. *Let $X_1, \ldots, X_n$ be the random variables at the leaves of a phylogenetic tree. Then, the joint probability of observing the nucleotide distribution $x_1, \ldots, x_n \in \Sigma$ at the leaves of the tree can be expressed as:*

$$\Pr(X_1 = x_1, \ldots, X_n = x_n) = \sum_{\text{states on } \Sigma \text{ of all internal nodes}} P_R(x_R) \prod_{e=(p \to c) \in E} M_e(x_p, x_c), \tag{9}$$

where $x_c = x_j$ for each leaf $j \in \{1, \ldots, n\}$.

We note that we are summing over all possible nucleotide distributions at the internal nodes of the tree when the distribution at the leaves is fixed, and the total number of configurations is $|\Sigma|^I = 4^I$. For each configuration, the probability is computed by multiplying the root probability with the conditional probabilities of transitioning from each parent state to each child state, where the child states at the leaves are fixed to be the considered $x_1, \ldots, x_n$. The total number of multiplications for each configuration is equal to the number of edges E.

2.2 Rooted Trivalent Trees and Computational Cost

In this section, we consider a specific type of phylogenetic tree, the rooted trivalent tree, as an example to determine the computational cost associated with expression (9).

A rooted tree is called *trivalent* if each internal node has degree 3, that is, the root has 3 children and all other internal nodes have 2 children. Let us express the number of edges and internal nodes of a rooted trivalent tree in terms of the number of leaves:

Let n be the number of leaves of a rooted trivalent tree, and I the number of internal vertices. For any rooted tree, the number of edges is $E = n+I-1$, and since it is a trivalent tree, $E = 2I+1$ (because each internal node has 2 edges that connect them to their corresponding children except for the root, which has an 3 children). This yields $I = n - 2$, so we get a **number of edges**:

$$E = 2n - 3. \tag{10}$$

If we now consider that the transition matrix M_e has k independent parameters for each edge e (depending on the chosen model), and that the root distribution has r independent parameters, then the **total number of parameters of the phylogenetic tree** is (assuming that the parameters for each edge are independent of those of all other edges):

$$kE + r = k(2n - 3) + r. \tag{11}$$

We can obtain the **computational cost of the joint probability** of a given nucleotide distribution at the leaves of a rooted trivalent tree by counting the number of operations in expression (9).

- Since the number of summands in the sum equals the total number of nucleotide configurations at the internal nodes of the tree for a fixed leaf distribution, we have $|\Sigma|^I = 4^{n-2}$ summands for a rooted trivalent tree.
- For a fixed nucleotide configuration at the internal nodes of the tree, the number of products is equal to $E = 2n - 3$.

Hence, the total number of multiplications is $|\Sigma|^I E$ and the total number of additions is $|\Sigma|^I - 1$. This yields a naive computational cost of the order;

$$O(|\Sigma|^I E) = O(|\Sigma|^n), \tag{12}$$

which is exponential in n.

The same order of computational cost—**exponential in the number of leaves**—is obtained for other rooted trees, such as the rooted binary tree, since the number of internal nodes I is linear in the number of leaves n. We observe that the computation using the explicit expression (9) is infeasible for large n. Nevertheless, there exist efficient alternatives for computing the joint probability, for instance, the *Felsenstein pruning algorithm*, which we introduce in the following section.

2.3 The Felsenstein Pruning Algorithm

In this section, we introduce the Felsenstein pruning algorithm, also referred to as the Forward algorithm on a tree, which computes the joint probability of observing a given nucleotide distribution at the leaves of a phylogenetic tree, with a computational cost that is linear in the number of leaves.

Let $X_1, \ldots, X_n$ be the random variables at the leaves of a rooted phylogenetic tree, and let $x_1, \ldots, x_n \in \Sigma$ be states. Let T be a phylogenetic tree, and let $v \in V(T)$ be a node of the tree,

with associated random variable X_v. Let T_v be the subgraph of T containing all descendants of v, the descendants of the descendants, and so on, down to a set of leaves labelled by indices $i_1, \ldots, i_k \in \{1, \ldots, n\}$, which we refer to as the descendant leaves of v. Let $O_v = (X_{i_1}, \ldots, X_{i_k})$ and $o_v = (x_{i_1}, \ldots, x_{i_k})$, where O stands for observable.

With this notation, we define:

Definition 11 (Forward variable)**.** We define the *forward variable* as the probability

$$L_v(x) = \Pr(O_v = o_v \mid X_v = x) \tag{13}$$

for a certain state $x \in \Sigma$.

Note:

- For a leaf $j \in \{1, \ldots, n\}$ we have $L_j(x) = \delta(x_j, x)$ where δ is the Dirac delta.
- For an internal node v with children c, since the descendant leaves of v are the descendant leaves of its children, we have:

$$\begin{aligned} L_v(x) &= \prod_c \Pr(O_c = o_c \mid X_v = x) = \prod_c \sum_{x_c \in \Sigma} \Pr(O_c = o_c \mid X_c = x_c) \Pr(X_c = x_c \mid X_v = x) \\ &= \prod_c \sum_{x_c \in \Sigma} L_c(x_c)\, M_e(x, x_c) \end{aligned} \tag{14}$$

  where we apply the law of total probability to the conditional probability given the event $\{X_v = x\}$, together with the Markov property of the process.

- We have the straightforward relation

$$\begin{aligned} \Pr(X_1 = x_1, \ldots, X_n = x_n) &= \sum_{x_R \in \Sigma} \Pr(X_1 = x_1, \ldots, X_n = x_n \mid X_R = x_R) \Pr(X_R = x_R) \\ &= \sum_{x_R \in \Sigma} P_R(x_R) L_R(x_R) \end{aligned} \tag{15}$$

  known as the *root likelihood*.

The idea behind the Felsenstein pruning algorithm is to use expression (15) to compute the joint probability by recursively obtaining the forward variables from the bottom of the tree (leaves) up to the top (root). Let us explicit this algorithm, and obtain its computational cost:

1. Initial case: Obtain $L_i(x)$ for the leaves $i = 1, \ldots, n$ and for states $x \in \Sigma$. No computational cost is involved here.
2. Recursion: We apply a bottom-up recursion from children to parents. That is, we recursively compute the forward variables of the interior nodes v for all states $x \in \Sigma$ from the forward variables of their children c, using expression (14), starting at the parents of the leaves and climbing the tree until reaching the root. Looking at expression (14), let us count the number of operations:
   - For each child of an interior node v, we are doing $|\Sigma|^2$ multiplications and $|\Sigma|^2 - 1$ additions, since we consider all $x, x_c \in \Sigma$.
   - For each interior node v, which we assume has $d = 2$ children (see Remark 9), we are doing $d|\Sigma|^2$ multiplications and $d(|\Sigma|^2 - 1)$ additions.
   - For all the I interior nodes in the tree, we are doing $dI|\Sigma|^2$ multiplications and $dI(|\Sigma|^2 - 1)$ additions.

3. Root likelihood: Once we obtain $L_R(x)$ for all $x \in \Sigma$, we use expression (15) to compute the desired probability, with additional $|\Sigma|$ multiplications and $|\Sigma| - 1$ additions.

From the previous counting, we have a total of $dI|\Sigma|^2 + |\Sigma|$ multiplications and $dI(|\Sigma|^2 - 1) + |\Sigma| - 1$ additions. Since $|\Sigma| = 4$ and $d = 2$ are constants, and the number of internal nodes I is linear in the number of leaves n, we obtain a computational cost

$$O(I) = O(n) \tag{16}$$

which is linear in n.

2.4 Branch Length, log-det Distance and Mutation Rate

In this final section, we introduce the concept of branch length for an edge of a phylogenetic tree and relate it to the log-det distance and the mutation rate in a continuous-time Markov model.

Let us start by defining the concept of branch length of an edge:

Definition 12 (Branch length)**.** Let e be an edge of a phylogenetic tree, with associated transition matrix M_e. Then, we define the *branch length* of the edge e as

$$l(e) = -\frac{1}{4} \log \det M_e. \tag{17}$$

We note that, for each edge e, the branch length coincides with the log-det distance of a substitution process with Markov matrix M_e.

We recall that the log-det distance is an additive mathematical distance, defined as

$$d_{\mathrm{LD}}(S_0, S_1) = -\frac{1}{4} \log \det M, \tag{18}$$

where S_0 is a nucleotide sequence and S_1 is its mutated version, obtained via a substitution process governed by the Markov matrix M.

This distance provides an asymptotic approximation of the empirical log-det distance between two sequences S_0 and S_1, defined as

$$d_{\mathrm{LD}}^{\mathrm{emp}}(S_0, S_1) = -\frac{1}{4} \log \det F, \tag{19}$$

where F is a 4×4 matrix whose entries are the relative frequencies of substitutions from S_0 to S_1.

When substitution processes along the edges are modelled as continuous in time, the transition matrices take the form

$$M(t) = \exp(Qt), \tag{20}$$

where Q is a rate matrix. In this continuous-time Markov model, we recall that the instantaneous mutation rate (the expected number of substitutions per unit time) is defined as

$$r = \sum_i \pi_i \sum_{j \neq i} Q_{ij}, \tag{21}$$

where π is the stationary distribution. Alternatively, since each row of the rate matrix Q sums to 0, the mutation rate can be expressed as

$$r = -\sum_i \pi_i Q_{ii}. \tag{22}$$

Let us relate the mutation rate r at each edge with the branch length l (i.e., the log-det distance $d_{\mathrm{LD}} = l$). If we assume that the transition matrices are of the form $M(t) = \exp(Qt)$, then, using the fact that $\det M(t) = \exp\bigl(t \operatorname{tr}(Q)\bigr)$, we obtain

$$l = -\frac{1}{4} \log \det M(t) = -\frac{t}{4} \operatorname{tr}(Q). \tag{23}$$

Since $-\operatorname{tr}(Q) = \sum_i (-Q_{ii})$, if the stationary distribution is uniform, that is, $\pi_i = \frac{1}{4}$, then we have

$$r = -\frac{1}{4} \operatorname{tr}(Q). \tag{24}$$

Hence, for a uniform stationary distribution,

$$l = rt, \tag{25}$$

that is, the branch length equals the product of the mutation rate and the time.